

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)**

**Факультет прикладной информатики
Образовательная программа Мобильные и сетевые технологии
Направление подготовки 09.03.03 Прикладная информатика**

**О Т Ч Е Т
о курсовой работе
«Мобильное приложение для отображения ленты новостей»**

Обучающийся:

Фадеев Дмитрий, К3439

Преподаватель:

Шатинский Григорий Сергеевич

Санкт-Петербург

2025

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	3
ВВЕДЕНИЕ	4
1 Обзор проекта	5
1.1 Назначение приложения	5
1.2 Контекст и ограничения	5
1.3 Ожидаемые результаты	6
1.4 Технологический стек	6
2 Требования и постановка задачи	8
2.1 Функциональные требования	8
2.2 Нефункциональные требования	8
2.3 Ограничения и допущения	9
2.4 Сценарии использования	9
3 Анализ предметной области	11
3.1 Типовые приложения класса	11
3.2 Пользовательские ожидания	11
3.3 Выводы для проектирования	11
4 Архитектура	13
5 Проектирование и моделирование	15
5.1 Навигация и экраны	15
5.2 Модель данных	15
5.3 Репозиторий и слой данных	15
6 Реализация	17
6.1 Точка входа и запрос разрешений	17
6.2 Постраничное обновление и результат синхронизации	18
6.3 Фоновая синхронизация и уведомления	19
ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	26

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

API – Application Programming Interface, интерфейс программирования приложений.

APK – Android Package, формат пакета приложения для Android.

DAO – Data Access Object, объект доступа к данным (в контексте Room – интерфейс для операций с БД).

FAB – Floating Action Button, плавающая кнопка действия.

Jetpack – набор библиотек и компонентов Android для упрощения разработки (Navigation, ViewModel, Room, WorkManager и др.).

KSP – Kotlin Symbol Processing, инструмент обработки аннотаций в проектах на Kotlin (используется компилятором Room).

MVVM – Model–View–ViewModel, архитектурный паттерн разделения представления, состояния и данных.

Room – библиотека Android Jetpack для работы с локальной реляционной базой данных SQLite.

UI – User Interface, пользовательский интерфейс.

ViewModel – компонент Jetpack для хранения и управления данными, связанными с экраном, с учётом жизненного цикла.

ВВЕДЕНИЕ

Курсовая работа посвящена разработке мобильного приложения «Chattersy» для платформы Android. Приложение представляет собой клиент новостной ленты с возможностью просмотра списка сообщений, получения данных из сетевого API, локального кэширования для работы без подключения к сети, ручного и фоновое обновления, а также интеграции с системой уведомлений и разрешений операционной системы.

Актуальность темы обусловлена широким распространением мобильных приложений класса «лента сообщений» и необходимостью владения современными средствами разработки под Android: языком Kotlin, компонентами Android Jetpack (Navigation, ViewModel, LiveData, Room, WorkManager), сетевыми библиотеками (Retrofit) и паттернами архитектуры (MVVM). Задача разработки такого приложения по этапам позволяет последовательно освоить навигацию, управление состоянием, работу с данными и фоновые процессы в условиях типовых ограничений учебного проекта.

Цель работы – реализовать функционирующее Android-приложение, удовлетворяющее заданному техническому заданию и демонстрирующее применение перечисленных технологий. В ходе работы решались задачи: организация навигации между экранами (Лента, Профиль, Настройки), внедрение архитектуры MVVM с сохранением состояния при смене конфигурации, интеграция с публичным API и локальной базой данных Room, реализация офлайн-режима и постраничного обновления данных, добавление фоновой синхронизации и уведомлений, запрос и использование разрешений (уведомления, контакты) в соответствии с политиками платформы.

1 Обзор проекта

1.1 Назначение приложения

В рамках курсовой работы разработано мобильное приложение «Chattersy» для платформы Android. Приложение относится к классу клиентов новостной ленты (лента сообщений) и демонстрирует типовые возможности: просмотр списка сообщений, получение данных из сетевого API, локальное кэширование для офлайн-доступа, ручное и фоновое обновление, управление разрешениями и уведомлениями.

Назначение приложения – учебно-демонстрационное: в нём последовательно реализованы навигация на основе Activity и Fragment, архитектура MVVM, работа с сетью и локальной базой данных Room, компоненты Material Design, фоновая синхронизация через WorkManager и интеграция с системой уведомлений и разрешений Android. Целевая аудитория – пользователи мобильных устройств под управлением Android, желающие просматривать ленту сообщений с возможностью работы без постоянного подключения к сети; в контексте курсовой работы приложение ориентировано на проверку преподавателем и демонстрацию владения технологиями Android (Kotlin, Jetpack, Room, Retrofit, WorkManager, система разрешений).

1.2 Контекст и ограничения

Разработка выполнялась по этапам в соответствии с техническим заданием. Использовался публичный тестовый API (JSONPlaceholder) для получения списка постов; структура данных и эндпоинты считались заданными. Целевая платформа – Android с минимальным уровнем API 24; учтены особенности Android 13+ в части запроса разрешения на показ уведомлений.

Ограничения включают использование только официальных компонентов Android и библиотек Android Jetpack без привлечения сторонних фреймворков для UI и архитектуры. Функциональность ограничена рамками

ТЗ: лента сообщений, профиль пользователя с редактированием имени и статуса, настройки (тема, офлайн-режим, управление разрешениями), без регистрации и серверной части.

1.3 Ожидаемые результаты

В результате разработки получено рабочее приложение, в котором реализованы: единая точка входа (MainActivity) с нижней навигацией по трём экранам (Лента, Профиль, Настройки), управление состоянием экранов через ViewModel с сохранением при смене конфигурации, загрузка и отображение списка сообщений из API с сохранением в локальную БД Room, офлайн-режим и ручное обновление, постраничная подгрузка данных, лайки сообщений с сохранением в БД, фоновая синхронизация по расписанию и при появлении сети, уведомления при успешном обновлении данных, запрос и использование разрешений (уведомления, контакты).

1.4 Технологический стек

Приложение построено на следующем технологическом стеке:

- язык программирования – Kotlin. Для сборки используется Gradle (Kotlin DSL); модуль приложения подключает плагины Android Application и KSP для генерации кода Room,
- компоненты Android Jetpack: Navigation Component (NavController, NavHostFragment, граф навигации) для переключения между экранами, ViewModel и LiveData для управления состоянием и реактивного обновления UI, Room для локальной базы данных, ViewBinding не используется – разметка подключается через findViewById и инфлейт layout-файлов,
- сетевой слой – библиотека Retrofit с конвертером Gson и клиентом OkHttp (настроены таймауты и при необходимости протокол HTTP/1.1). Данные запрашиваются корутинами (suspend-функции),
- фоновая обработка – WorkManager для периодической и одноразовой синхронизации. Уведомления реализованы через классы Android API (NotificationChannel, NotificationCompat, PendingIntent); запрос

разрешений – через Activity Result API (`RequestMultiplePermissions`).

Дополнительно используются: Glide для загрузки изображений аватаров, Material Design компоненты (`BottomNavigationView`, `Toolbar`, `FloatingActionButton`, `SwipeRefreshLayout`, `MaterialCardView`, `SwitchMaterial`), `ConnectivityManager` и `NetworkCallback` для отслеживания доступности сети. Логирование выполняется через `Log.d/Log.e` с единым тегом по компонентам.

2 Требования и постановка задачи

2.1 Функциональные требования

Функциональные требования к приложению заданы техническим заданием по четырём этапам.

Первый этап: одна главная активность как точка входа; навигация с помощью Bottom Navigation; три отдельных экрана – новостная лента (изначально заглушка), профиль пользователя, настройки; навигация через NavController и Navigation Graph; отслеживание жизненного цикла компонентов с выводом в лог.

Второй этап: наличие ViewModel для профиля и настроек; экран «Профиль» – отображение и редактирование имени и статуса с сохранением во ViewModel; экран «Настройки» – переключатель темы (светлая/тёмная), состояние во ViewModel; сохранение данных при повороте экрана; логирование жизненного цикла ViewModel.

Третий этап: MessageRepository, получающий данные из публичного API; Retrofit-клиент с корутинами; отображение списка сообщений на вкладке «Лента» с помощью RecyclerView; сохранение данных в локальную базу Room; при отсутствии сети – загрузка сообщений из базы; кнопка «Обновить» для ручного обновления.

Четвёртый этап: кастомный вид элемента списка (аватарка, имя, текст); применение Material Design (FloatingActionButton, AppBarLayout); возможность ставить лайк сообщению (иконка меняется при клике); настройка WorkManager для периодического обновления списка из сети; уведомление при успешной синхронизации с текстом «Новые данные получены»; запрос разрешений при запуске (уведомления, контакты и т.п.); уведомления в фоне и реакция на системные события (онлайн/офлайн).

2.2 Нефункциональные требования

Код должен быть структурирован и поддерживать дальнейшее расширение; интерфейс – понятным. Применяются принципы разделения

ответственности между UI и ViewModel и реактивное обновление UI через LiveData. Сетевые операции выполняются только через корутины; работа с базой – через DAO и сущности Room; репозиторий выступает единой точкой взаимодействия ViewModel с данными. Код оформлен в соответствии с принципами MVVM.

2.3 Ограничения и допущения

Приняты следующие ограничения и допущения: использование только официального SDK и Jetpack-библиотек в рамках заданного стека; формат и структура данных API считаются неизменными (JSONPlaceholder или аналог); минимальная поддерживаемая версия Android – API 24; для Android 13+ учитывается необходимость запроса разрешения POST_NOTIFICATIONS; допускается ручное управление разрешениями через системные настройки (кнопка «Открыть настройки разрешений» в приложении). Серверная часть и аутентификация пользователей не входят в постановку задачи.

2.4 Сценарии использования

Основные сценарии использования приложения охватывают:

- первый запуск с запросом разрешений и отображением стартового экрана (Лента),
- переключение между вкладками Лента, Профиль, Настройки,
- просмотр ленты сообщений при наличии сети и при её отсутствии (данные из кэша),
- ручное обновление ленты (свайп или FAB) с индикацией загрузки и при актуальных данных – уведомлением «Данные актуальны»,
- постановку и снятие лайка с сохранением состояния,
- редактирование имени и статуса в профиле,
- переключение темы и офлайн-режима в настройках,
- открытие системных настроек разрешений приложения,
- получение уведомления о новых данных после фоновой синхронизации и открытие приложения по нажатию на уведомление,

– отображение количества контактов на экране «Профиль» при выданном разрешении.

3 Анализ предметной области

3.1 Типовые приложения класса

Приложения класса «лента сообщений» (новостная лента, микроблоги, агрегаторы постов) характеризуются следующими типовыми чертами: список элементов с прокруткой (часто с подгрузкой по мере скролла или постранично), отображение авторской информации (аватар, имя) и контента (текст, медиа), интерактивность (лайки, комментарии), обновление данных из сети с кэшированием для офлайн-просмотра, уведомления о новых записях.

3.2 Пользовательские ожидания

Пользователь ожидает от такого приложения стабильное отображение ленты, возможность обновить данные вручную и получить обратную связь (индикатор загрузки, сообщение об актуальности или об ошибке). При отсутствии сети ожидается показ ранее загруженных данных. Уведомления не должны показываться без необходимости (например, при отсутствии новых данных). Запрос разрешений должен быть контекстно уместен (при первом появлении экрана или при обращении к функции, требующей разрешения). В приложении эти ожидания учтены за счёт постраничной загрузки с критерием «страница совпадает с последними сообщениями», отдельной логики для ручного обновления (Toast «Данные актуальны») и фоновой синхронизации (уведомление только при `!wasUpToDate`), запроса разрешений в `onResume()` с флагом «один раз за сессию».

3.3 Выводы для проектирования

Из анализа предметной области следуют выводы для проектирования: единый репозиторий как источник данных для UI с чётким разделением «сеть / локальная БД»; реактивная модель (Flow/LiveData) для автоматического обновления экрана при изменении данных; явная обработка состояний сети и офлайн-режима; вынос логики разрешений и уведомлений в отдельные модули (папка `notifications`); использование стандартных компонентов Android (Navigation, WorkManager, Activity Result API) для снижения рисков

несовместимости и упрощения поддержки.

4 Архитектура

Приложение построено по многослойной архитектуре с разделением на уровень представления (UI), уровень бизнес-логики и состояния (ViewModel) и уровень данных (Repository, источники данных).

Уровень представления: единственная активность MainActivity является точкой входа и контейнером для навигации; экраны реализованы фрагментами (NewsFeedFragment, ProfileFragment, SettingsFragment), размещаемыми в NavHostFragment. Навигация задаётся графом (nav_graph.xml) и управляется NavController; BottomNavigationView синхронизируется с графом через setupWithNavController. Фрагменты не содержат бизнес-логики: они подписываются на LiveData из ViewModel и вызывают методы ViewModel при действиях пользователя.

Уровень состояния и координации: для экранов ленты, профиля и настроек используются соответственно NewsFeedViewModel, ProfileViewModel, SettingsViewModel. ViewModel получают данные и выполняют операции через репозитории (или напрямую через предпочтения/контекст там, где репозиторий не задействован). Жизненный цикл корутин привязан к viewModelScope, чтобы при уничтожении ViewModel отменялись незавершённые операции.

Уровень данных: MessageRepository инкапсулирует доступ к сетевым данным (Retrofit, MessageApiService) и к локальной БД (Room, MessageDao). Репозиторий определяет доступность сети (ConnectivityManager, учёт офлайн-режима в SharedPreferences), выполняет постраничную загрузку и возвращает RefreshResult(wasUpToDate) для согласования логики уведомлений и сообщения «Данные актуальны». Класс ChattersyApplication регистрирует NetworkCallback и при появлении сети ставит в очередь WorkManager одноразовую работу SyncWorker. Периодическая работа SyncWorker планируется в

`MainActivity.onCreate()`. Логика запроса разрешений и проверки их наличия вынесена в объект `AppPermissions` (папка `notifications`); использование контактов – в `ContactsHelper`.

На рисунке 1 приведена упрощённая схема компонентов и потоков данных.

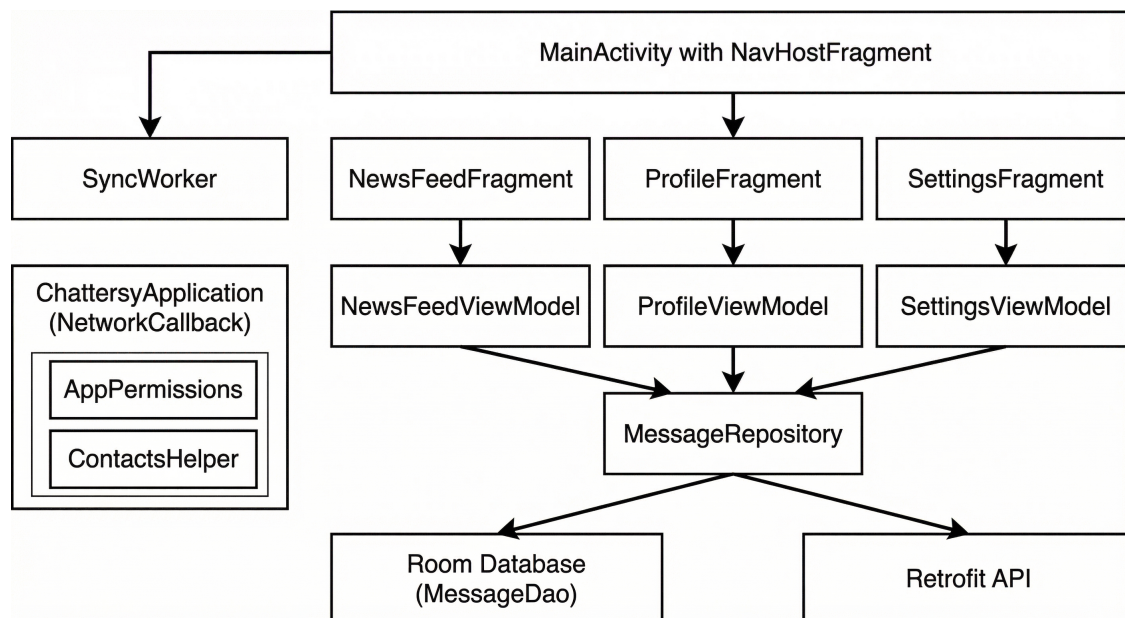


Рисунок 1 – Схема архитектуры приложения

5 Проектирование и моделирование

5.1 Навигация и экраны

Навигационная модель приложения – один граф с тремя направлениями (destinations). Стартовый экран – лента сообщений (newsFeedFragment). Граф описан в XML и подключён к NavHostFragment в разметке активности; переключение вкладок в BottomNavigationView приводит к вызову NavController.navigate() к соответствующему идентификатору фрагмента. Отдельные экраны не передают друг другу параметры через аргументы навигации в рамках базового сценария; состояние профиля и настроек хранится во ViewModel (и при необходимости в SavedStateHandle).

5.2 Модель данных

Модель данных ленты представлена сущностью Message с полями: id, avatarUrl, userName, text, likesCount, isLiked. Сущность используется и как модель для отображения, и как Entity Room (таблица messages). Сетевой контракт API описан через DTO PostApiResponse; маппинг в Message выполняется функцией расширения toMessage() с учётом уже сохранённого состояния лайка из БД. Для результата обновления ленты введён тип RefreshResult (wasUpToDate: Boolean).

5.3 Репозиторий и слой данных

Репозиторий сообщений предоставляет поток данных из БД (getMessages(): Flow<List<Message>>) и методы refreshMessages(): RefreshResult, loadInitialMessages(), likeMessage(id). При обновлении выполняется постраничный запрос к API (размер страницы 20); полученные страницы мержаются в БД через insertAll(REPLACE). Цикл загрузки прекращается при пустой странице, при совпадении множества id текущей страницы с множеством id последних 20 сообщений в БД или при неполной странице. В зависимости от этого формируется RefreshResult.wasUpToDate. Проверка сети выполняется

до начала обновления; при включённом офлайн-режиме в настройках сетевые запросы не выполняются.

6 Реализация

6.1 Точка входа и запрос разрешений

Главная активность настраивает навигацию и планирует периодическую работу `WorkManager` в `onCreate()`. Запрос разрешений выполняется в `onResume()` через вспомогательный метод: получается массив недостающих разрешений из `AppPermissions.permissionsToRequest(this)` при непустом массиве и при условии что запрос ещё не выполнялся в текущей сессии, вызывается `requestPermissionsLauncher.launch(toRequest)`. Используется контракт `RequestMultiplePermissions`, что позволяет запросить разрешения на уведомления (Android 13+) и на доступ к контактам одним диалогом.

Листинг 1 – Настройка навигации в MainActivity

```
val navHostFragment = supportFragmentManager
    .findFragmentById(R.id.nav_host_fragment) as NavHostFragment
val navController = navHostFragment.navController
val bottomNavigationView = findViewById<BottomNavigationView>(
    R.id.bottom_navigation)
bottomNavigationView.setupWithNavController(navController)
```

Листинг 2 – Сущность Message (Room Entity)

```
@Entity(tableName = "messages")
data class Message(
    @PrimaryKey val id: Int,
    val avatarUrl: String,
    val userName: String,
    val text: String,
    val likesCount: Int,
    val isLiked: Boolean
)
```

Листинг 3 – Интерфейс MessageDao

```
@Dao
interface MessageDao {
    @Query("SELECT * FROM messages ORDER BY id DESC")
    fun getAllMessages(): Flow<List<Message>>
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(messages: List<Message>)
```

```

        @Query("UPDATE messages SET isLiked = :isLiked, likesCount = :likesCount")
        suspend fun updateLikeStatus(id: Int, isLiked: Boolean, likesCount: Int) {
            // ...
        }
    }
}

```

Листинг 4 – Формирование списка разрешений для запроса (AppPermissions)

```

fun permissionsToRequest(activity: FragmentActivity): Array<String> {
    val list = mutableListOf<String>()
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        if (ContextCompat.checkSelfPermission(activity,
            Manifest.permission.POST_NOTIFICATIONS)
            != PackageManager.PERMISSION_GRANTED) {
            list.add(Manifest.permission.POST_NOTIFICATIONS)
        }
    }
    if (ContextCompat.checkSelfPermission(activity,
        Manifest.permission.READ_CONTACTS)
        != PackageManager.PERMISSION_GRANTED) {
        list.add(Manifest.permission.READ_CONTACTS)
    }
    return list.toArray()
}

```

Листинг 5 – Планирование периодической работы WorkManager в MainActivity

```

private fun scheduleSyncWork() {
    val constraints = Constraints.Builder()
        .setRequiredNetworkType(NetworkType.CONNECTED)
        .build()
    val request = PeriodicWorkRequestBuilder<SyncWorker>(
        15, TimeUnit.MINUTES).setConstraints(constraints).build()
    WorkManager.getInstance(this).enqueueUniquePeriodicWork(
        SYNC_WORK_NAME, ExistingPeriodicWorkPolicy.KEEP, request)
}

override fun onResume() {
    super.onResume()
    requestAppPermissionsIfNeeded()
}

```

6.2 Постраничное обновление и результат синхронизации

В репозитории реализована постраничная загрузка с размером

страницы 20. Перед циклом запросов из БД получается список существующих сообщений и формируется множество id последних 20 записей (`existingLastIds`). В цикле вызывается `apiService.getMessage(start, PAGE_SIZE)`; ответ маппится в `Message` с учётом сохранённых лайков и вставляется в БД. Если множество id текущей страницы совпадает с `existingLastIds`, устанавливается `wasUpToDate = true` и цикл прерывается; аналогично при пустой или неполной странице. Метод возвращает `RefreshResult(wasUpToDate)`.

Листинг 6 – Фрагмент `MessageRepository`: цикл постраничной загрузки и критерий актуальности

```
val existingLastIds = if (existing.size >= PAGE_SIZE)
    existing.takeLast(PAGE_SIZE).map { it.id }.toSet()
else existing.map { it.id }.toSet()
var start = 0
var wasUpToDate = false
while (true) {
    val chunk = apiService.getMessage(start = start, limit = PAGE_SIZE)
    if (chunk.isEmpty()) { wasUpToDate = true; break }
    messageDao.insertAll(messages)
    val chunkIds = chunk.map { it.id }.toSet()
    if (existingLastIds.isNotEmpty() && chunkIds == existingLastIds) {
        wasUpToDate = true; break
    }
    if (chunk.size < PAGE_SIZE) { wasUpToDate = true; break }
    start += chunk.size
}
return RefreshResult(wasUpToDate = wasUpToDate)
```

6.3 Фоновая синхронизация и уведомления

Фоновая синхронизация реализована с помощью `WorkManager`: периодическая работа планируется в `MainActivity.onCreate()` через `enqueueUniquePeriodicWork` с интервалом 15 минут и ограничением `NetworkType.CONNECTED`; при появлении сети класс `ChattersyApplication` регистрирует `NetworkCallback` и ставит в очередь одноразовую работу `SyncWorker`, чтобы данные

обновились без ожидания следующего интервала. Worker наследует CoroutineWorker, в doWork() на Dispatchers.IO создаётся MessageRepository, вызывается repository.refreshMessages(); по возвращённому RefreshResult(wasUpToDate) решение о показе уведомления принимается только при !result.wasUpToDate, чтобы не уведомлять пользователя при отсутствии новых данных. Уведомление формируется в методе showNotification(): для Android 8+ создаётся NotificationChannel с идентификатором и именем из строковых ресурсов и важностью IMPORTANCE_DEFAULT; PendingIntent на MainActivity с флагами FLAG_ACTIVITY_NEW_TASK и FLAG_ACTIVITY_CLEAR_TOP позволяет открыть приложение по нажатию; NotificationCompat.Builder задаёт иконку, заголовок и текст из ресурсов, setAutoCancel(true); вызов NotificationManager.notify() обёрнут в try/catch (SecurityException на случай отсутствия разрешения POST_NOTIFICATIONS на Android 13+).

Листинг 7 – SyncWorker: выполнение синхронизации и показ уведомления

```
override suspend fun doWork(): Result = withContext(Dispatchers.IO) {
    try {
        val repository = MessageRepository(applicationContext)
        val result = repository.refreshMessages()
        if (!result.wasUpToDate) {
            showNotification(
                applicationContext.getString(R.string.notification_sync),
                applicationContext.getString(R.string.notification_sync)
            )
            Result.success()
        } catch (e: Exception) {
            Result.failure()
        }
    }
}

private fun showNotification(title: String, text: String) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val channel = NotificationChannel(channelId, channelName,
```

```

        NotificationManager.IMPORTANCE_DEFAULT)
    manager.createNotificationChannel(channel)
}
val pendingIntent = PendingIntent.getActivity(applicationContext,
    0, Intent(applicationContext, MainActivity::class.java).apply {
        flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TOP
    }, PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE)
val notification = NotificationCompat.Builder(applicationContext, channelId)
    .setSmallIcon(android.R.drawable.ic_menu_rotate)
    .setContentTitle(title).setContentText(text)
    .setContentIntent(pendingIntent).setAutoCancel(true).build()
try { manager.notify(NOTIFICATION_ID, notification) }
catch (_: SecurityException) { }
}

```

На рисунках ниже приведены скриншоты экрана ленты сообщений и экрана профиля приложения (файлы размещаются в папке images).

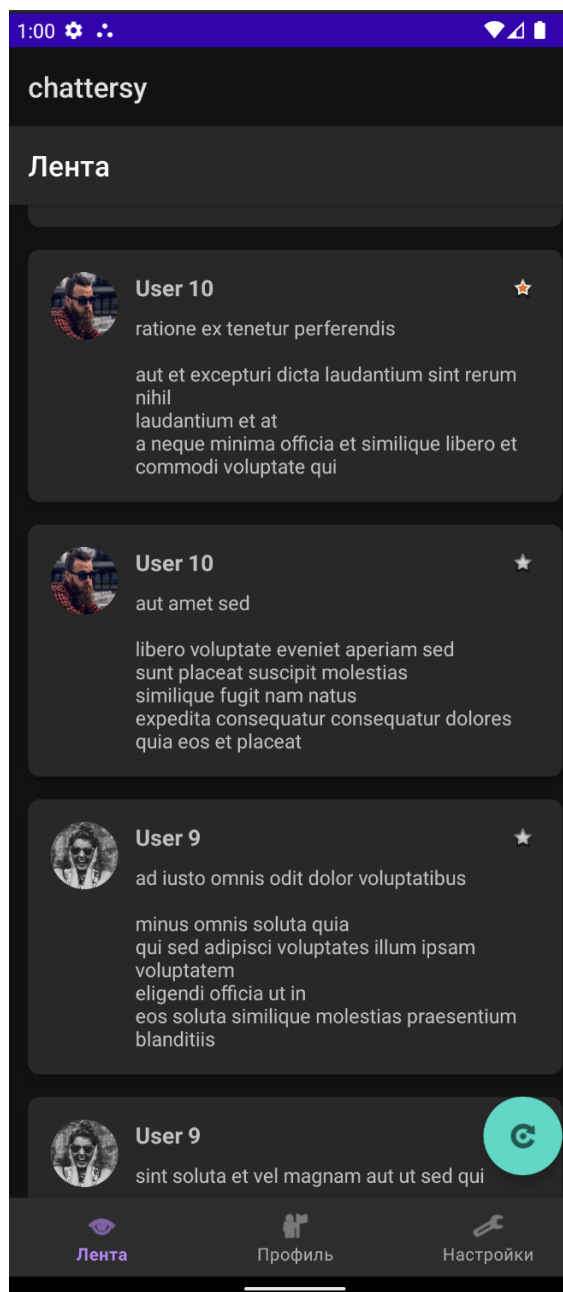


Рисунок 2 – Экран «Лента» приложения Chattersy

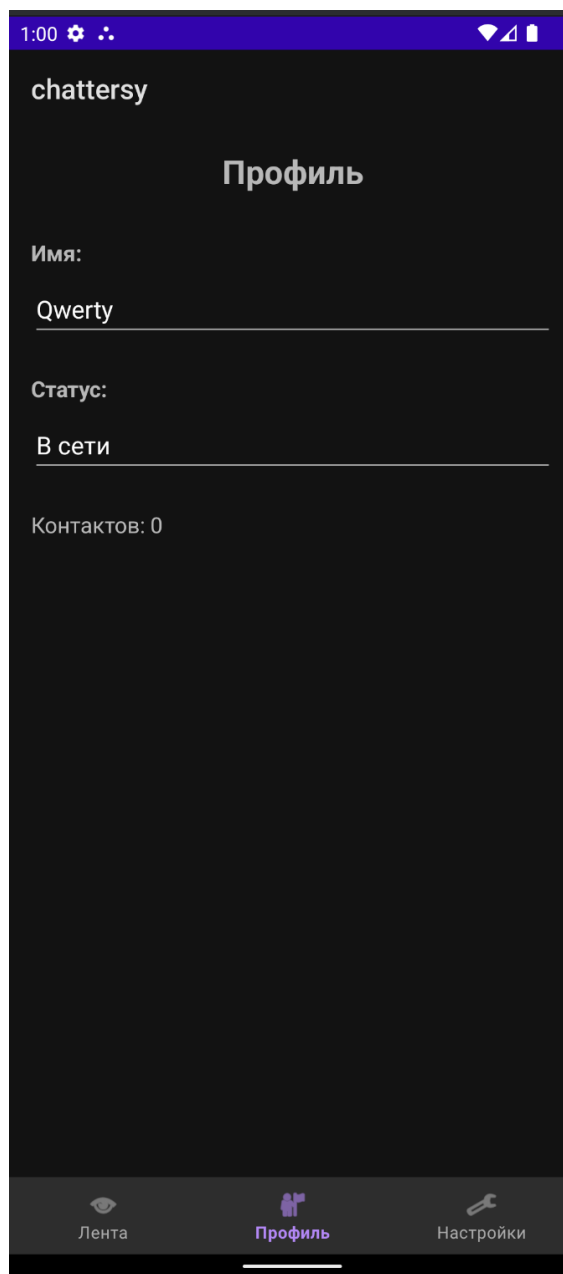


Рисунок 3 – Экран «Профиль» приложения Chattersy

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы разработано мобильное приложение «Chattersy» для платформы Android, реализующее функциональность новостной ленты с загрузкой данных из сетевого API, локальным кэшированием в базе данных Room, ручным и фоновым обновлением, уведомлениями и запросом системных разрешений. Все пункты технического задания по четырём этапам выполнены: обеспечена навигация на основе одной активности и трёх фрагментов с использованием NavController и Navigation Graph, реализовано управление состоянием экранов Профиль и Настройки через ViewModel с сохранением при повороте экрана, реализованы загрузка и отображение списка сообщений через RecyclerView, сохранение данных в Room и работа в офлайн-режиме, постраничное обновление с критерием актуальности данных, лайки сообщений, фоновая синхронизация через WorkManager и уведомления при появлении новых данных, запрос разрешений на уведомления и доступ к контактам с использованием их в интерфейсе (отображение количества контактов в профиле).

Применены современные средства разработки под Android: язык Kotlin, компоненты Jetpack (Navigation, ViewModel, LiveData, Room, WorkManager), библиотеки Retrofit и Glide, Material Design компоненты. Архитектура приложения соответствует принципам MVVM: слой представления (Activity, Fragment) отделён от логики и данных (ViewModel, Repository); репозиторий выступает единой точкой доступа к сетевым и локальным данным. Обработка ошибок сети, таймаутов и офлайн-режима реализована с отображением пользователю понятных сообщений; уведомления показываются только при фактическом появлении новых данных в фоновой синхронизации, а при ручном обновлении с уже актуальными данными выводится сообщение «Данные актуальны».

Дальнейшее развитие проекта может включать: расширение модели

данных (комментарии, вложения), аутентификацию пользователя, доработку UI (бесконечный скролл, превью медиа), тестирование (unit- и UI-тесты), настройку подписок на уведомления по типам контента. Разработанное приложение и сопроводительная документация в репозитории позволяют воспроизвести сборку, установку и проверку описанных сценариев использования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Android Developers. Documentation [Электронный ресурс]. – URL: <https://developer.android.com> (дата обращения: 29 декабря 2024).
2. Android Developers. Guide to app architecture [Электронный ресурс]. – URL: <https://developer.android.com/topic/architecture> (дата обращения: 29 декабря 2024).
3. Android Developers. WorkManager [Электронный ресурс]. – URL: <https://developer.android.com/topic/libraries/architecture/workmanager> (дата обращения: 29 декабря 2024).
4. Kotlin Language Documentation [Электронный ресурс]. – URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 29 декабря 2024).
5. Android Developers. Save data in a local database using Room [Электронный ресурс]. – URL: <https://developer.android.com/training/data-storage/room> (дата обращения: 29 декабря 2024).
6. Square. Retrofit. A type-safe HTTP client for Android and Java [Электронный ресурс]. – URL: <https://square.github.io/retrofit/> (дата обращения: 29 декабря 2024).
7. Material Design. Components [Электронный ресурс]. – URL: <https://m3.material.io/components> (дата обращения: 29 декабря 2024).